PACE UNIVERSITY

SEIDENBERG SCHOOL OF CSIS

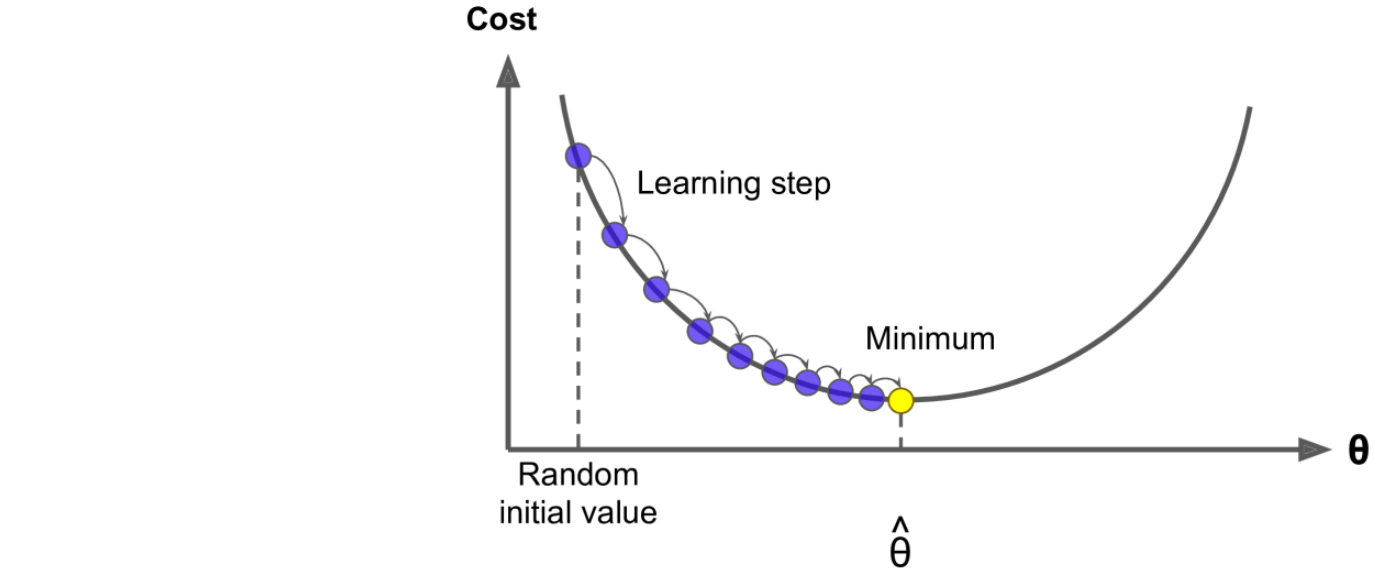
Stochastic Gradient Descent

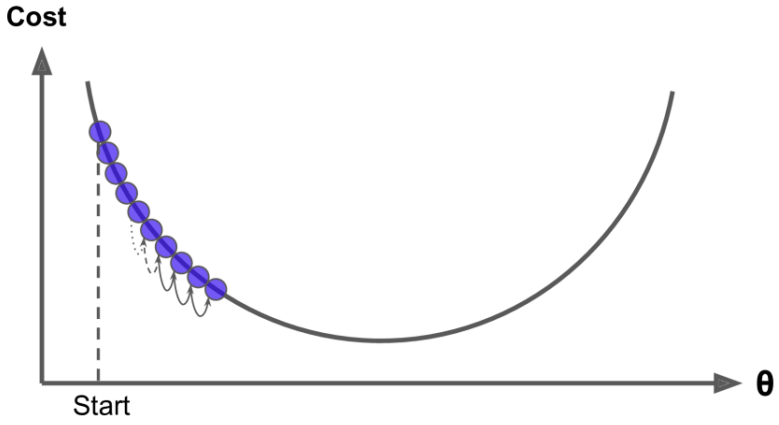
Dr. Krishna Bathula

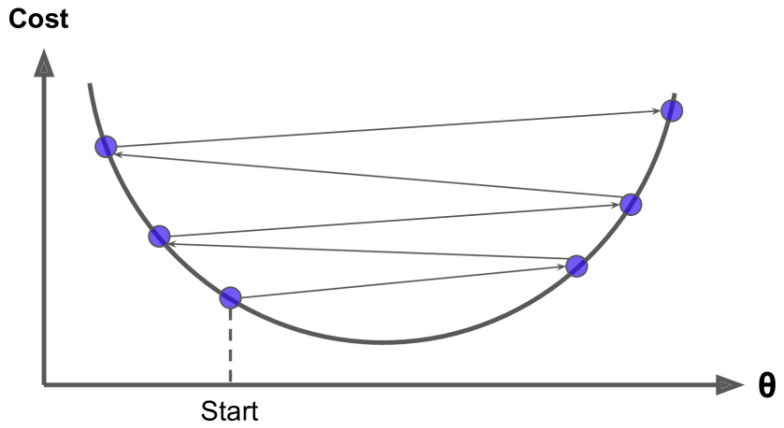
Learning Objectives

1. Gradient Descent Retrospective
2. Learning Rate revisiting
3. Batch Gradient Descent
4. **Stochastic Gradient Descent**
5. Mini-Batch Gradient Descent
6. Classifiers
7. Evaluation Metrics for Classification

Gradient Descent Retrospective







Gradient Descent Retrospective

Gradient descent is an **iterative** algorithm. $$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

We start with an **initial value**.

Then, at each iteration, we take a step in the direction of the negative of the gradient at the current point.

Gradient Descent is the **vector of partial derivatives**

$$\frac{\partial}{\partial \theta_j} \text{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

Batch Gradient Descent

For Gradient Descent, the equation for partial derivatives of the cost function is given as

$$\frac{\partial}{\partial \theta_j}\text{MSE}(\boldsymbol{\theta}) = \frac{2}{m}\sum_{i=1}^{m}\left(\boldsymbol{\theta}^T\mathbf{x}^{(i)} - y^{(i)}\right)x_j^{(i)}$$

Instead of computing these partial derivatives individually, we can compute them all simultaneously.

$$\nabla_{\boldsymbol{\theta}}\text{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0}\text{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1}\text{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n}\text{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m}\mathbf{X}^T(\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

The gradient vector, noted $\nabla_{\boldsymbol{\theta}}\text{MSE}(\theta)$, contains all the partial derivatives of the cost function.

Batch Gradient Descent

Batch Gradient Descent computes full training set X, at each Gradient Descent step.

That is, it uses the whole batch of training data at every step.

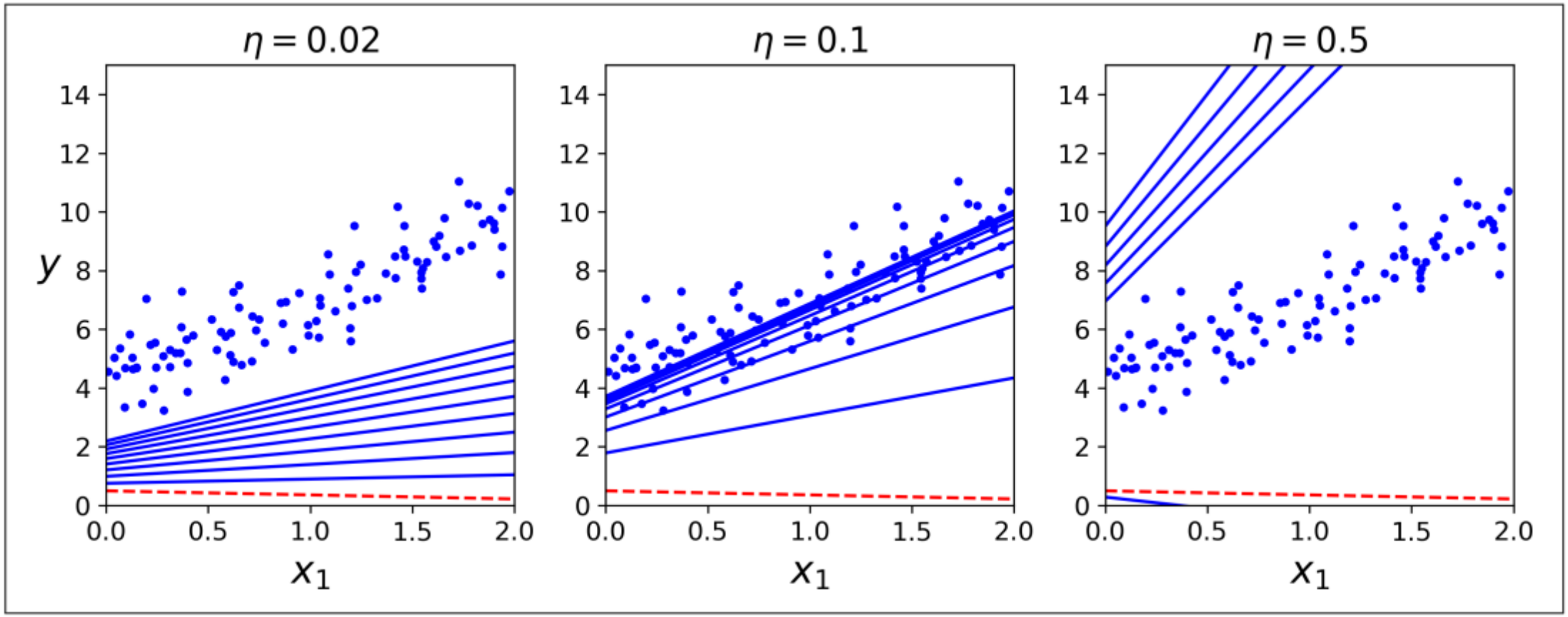
As a result it is terribly slow on very large training sets.

Once you have the gradient vector, which points uphill, just go in the opposite direction to go downhill. This means subtracting ∇θMSE(θ) from θ.

The gradient vector is multiplied by the learning rate η to determine the size of the downhill step.

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$$

Different Learning Rates



To find a good learning rate, you can use a grid search

Grid Search

A machine learning model has multiple hyperparameters set before the training of an ML model. These hyperparameters control the accuracy of the model.

For example:

- Learning rate in Gradient Descent
- Number of Estimators

But how to set the correct values for these hyperparameters?

Grid Search is a tuning technique that attempts to compute the optimum values of hyperparameters.

It is an exhaustive search that is performed on the specific parameter values of a model to be optimal.

Gradient Descent - Limitations

Gradient Descent sometimes gets **stuck** in a **local minimum** (non-uniform cost function) instead of finding the global minimum.

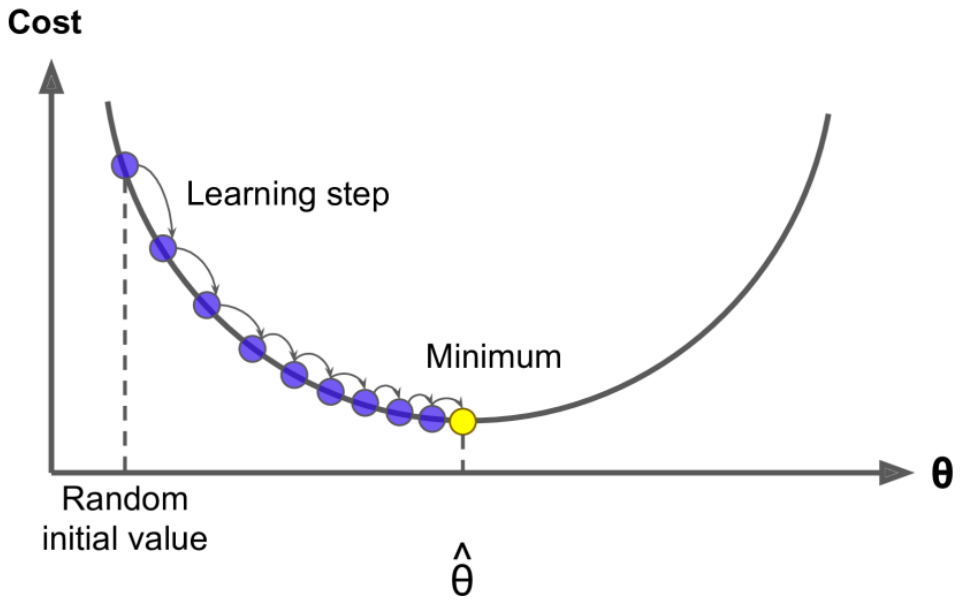
The main problem with **Batch Gradient Descent** is that it uses the **whole training set** to compute the gradients at every step.

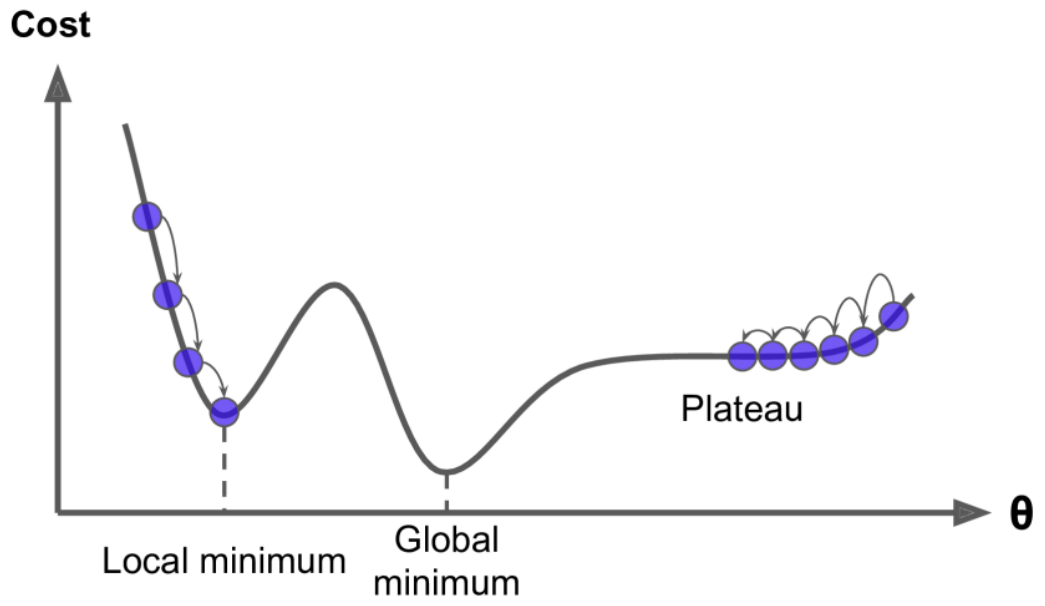
Another problem is the gradients that we calculate can become very **noisy**.

These **noisy gradients** are **just estimates** and not true gradients that point towards the highest rate of increase of the loss function.

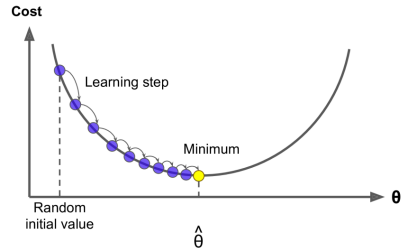
It **takes too long** for Gradient Descent on a large training sample **to converge**.

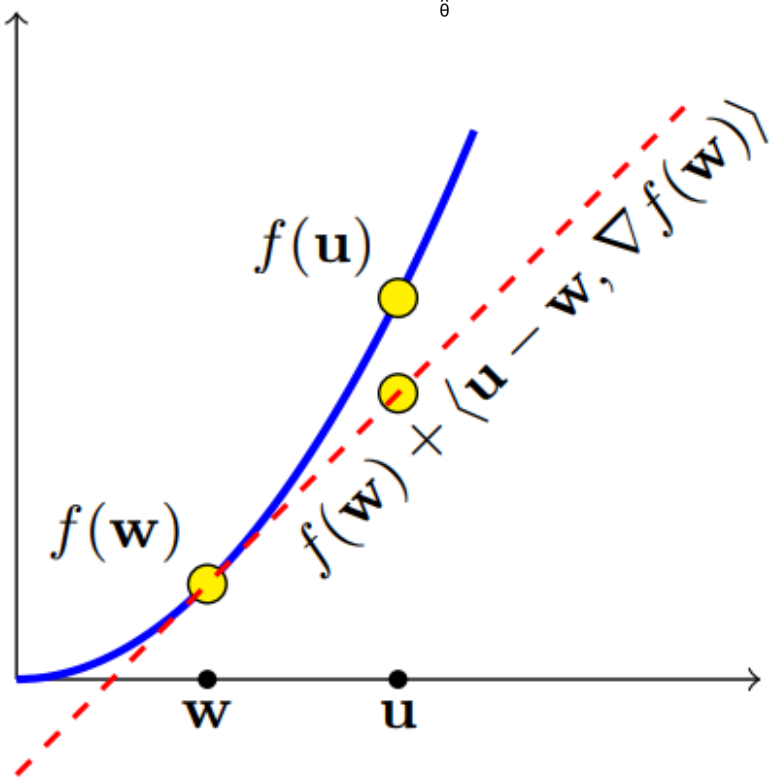
Gradient Descent Retrospective

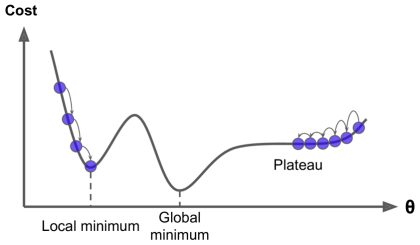


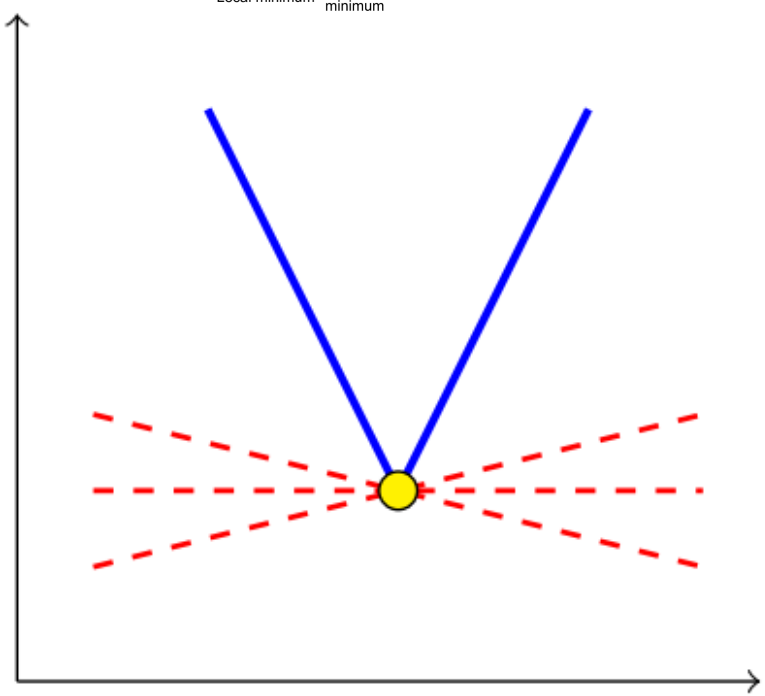


Gradient Descent Retrospective









Stochastic Gradient Descent

Stochastic gradient descent **SGD** is an optimization algorithm often used in machine learning applications to find the **model parameters** that correspond to the **best fit** between **predicted and actual outputs**.

SGD just picks a **random instance** in the training set at every step and computes the gradients based only on that **single instance**.

This **makes the algorithm much faster** since it has very little data to manipulate at every iteration.

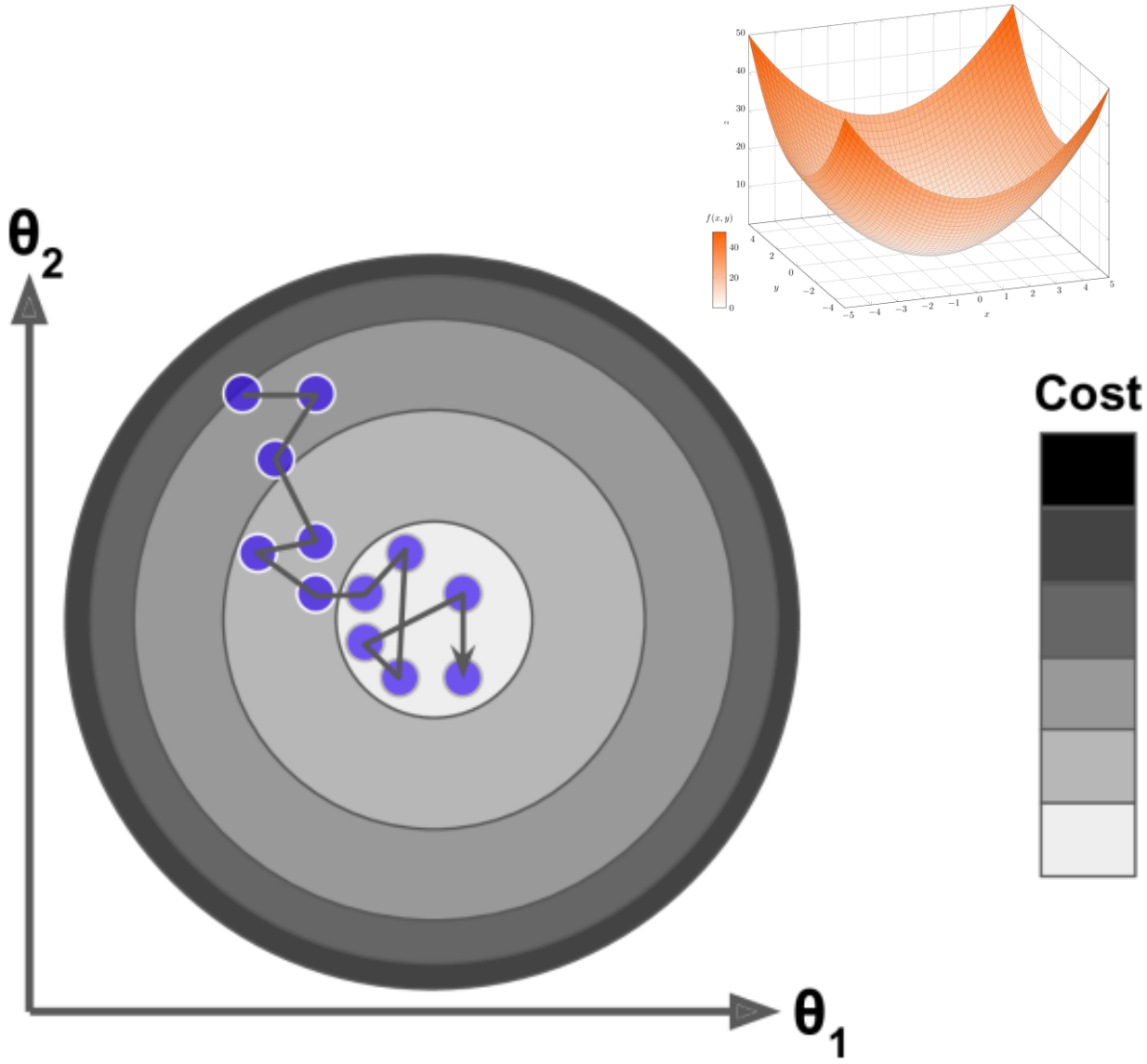
It also makes it possible to train on huge training sets, since only **one instance needs to be in memory at each iteration**

Stochastic Gradient Descent

- Instead of gently decreasing until it reaches the minimum, **the cost function will bounce up and down**, decreasing only on average.

- Over time it will end up **very close to the minimum**, but once it gets there it will continue to bounce around, **never settling**.

- So once the algorithm stops, the **final parameter values are good**, **but not optimal**.



Stochastic Gradient Descent

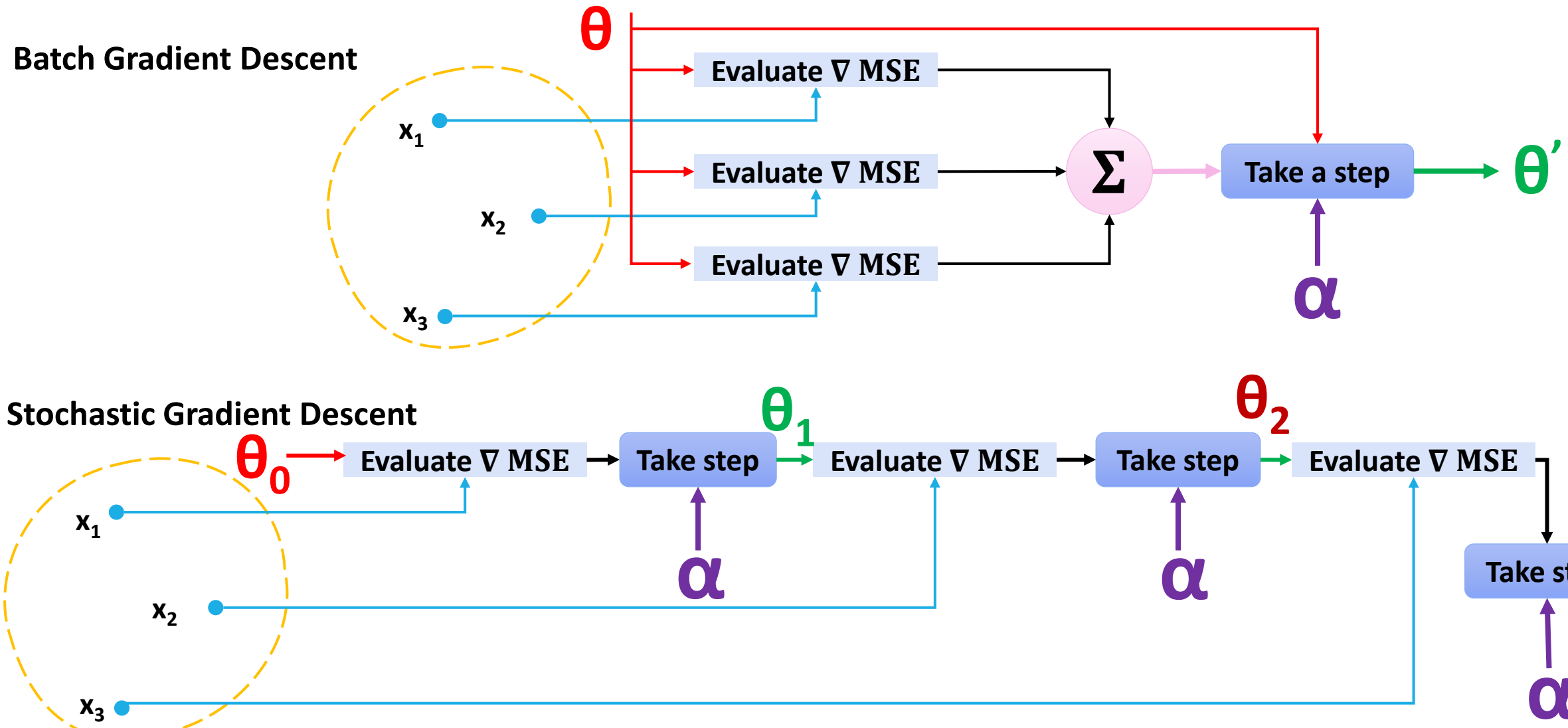
When using SGD, the training instances must be independent and identically distributed (IID), to ensure that the parameters get pulled towards the **global optimum**, on average.

A simple way to ensure this is to **shuffle the instances** during training

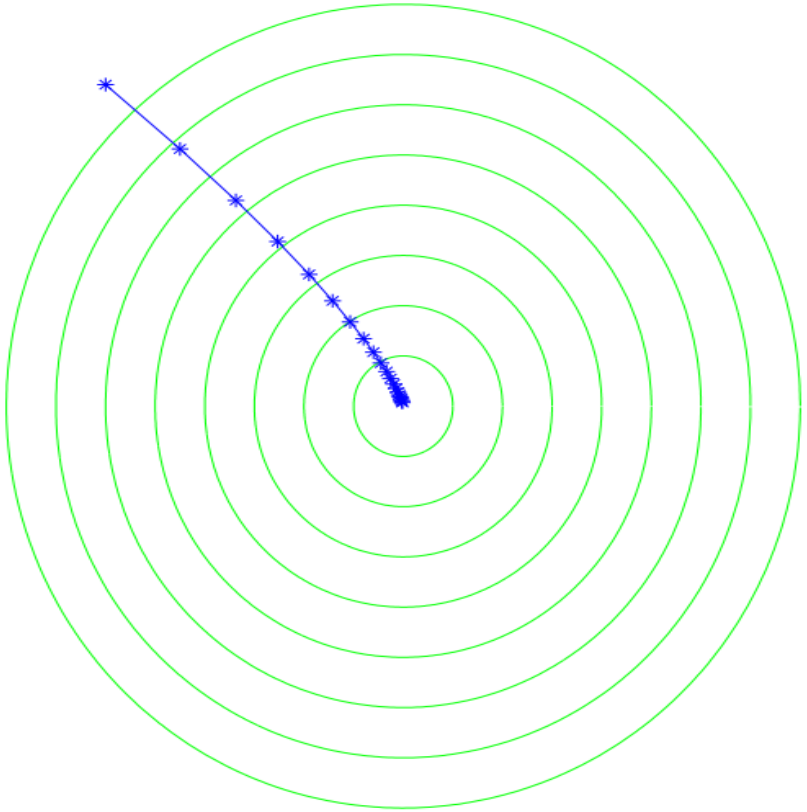
If you do not do this, for example, if the instances are sorted by label, SGD will start by **optimizing for one label**, then the next, and so on, and it will not settle close to the global minimum.

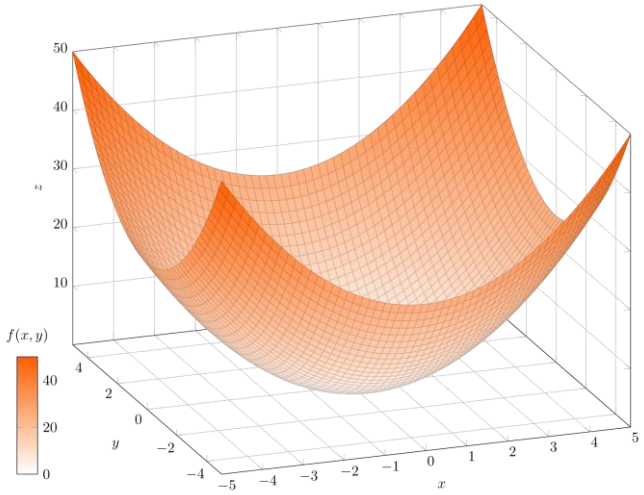
SGD has been successfully applied to **large-scale** and **sparse** machine learning problems

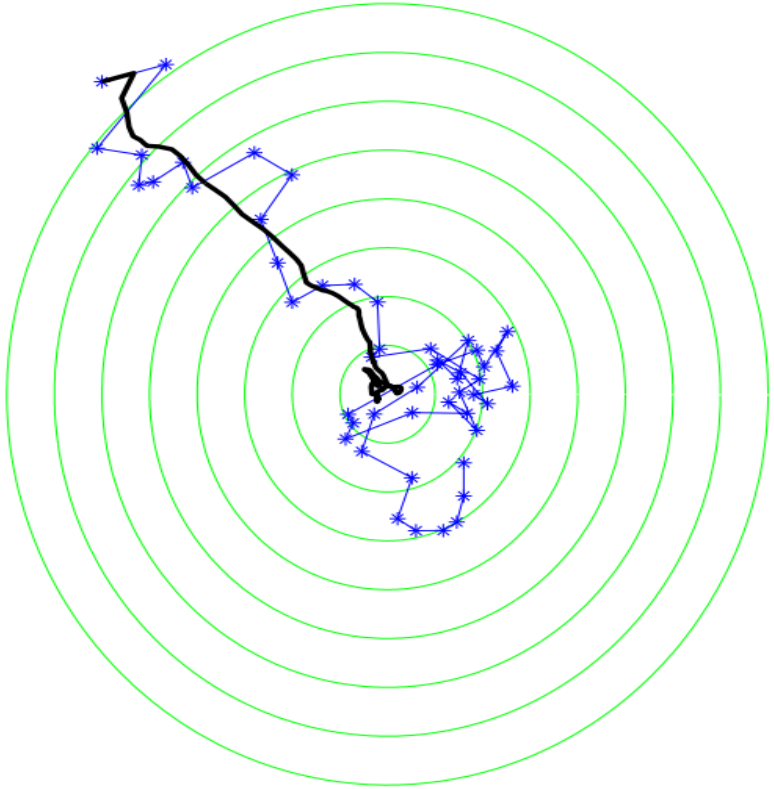
Batch Gradient Descent vs. Stochastic Gradient Descent



Gradient Descent vs. Stochastic Gradient Descent







Gradient Descent

Stochastic Gradient Descent

Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) for minimizing

$$f(\mathbf{w})$$

parameters: Scalar $\eta > 0$, integer $T > 0$

initialize: $\mathbf{w}^{(1)} = \mathbf{0}$

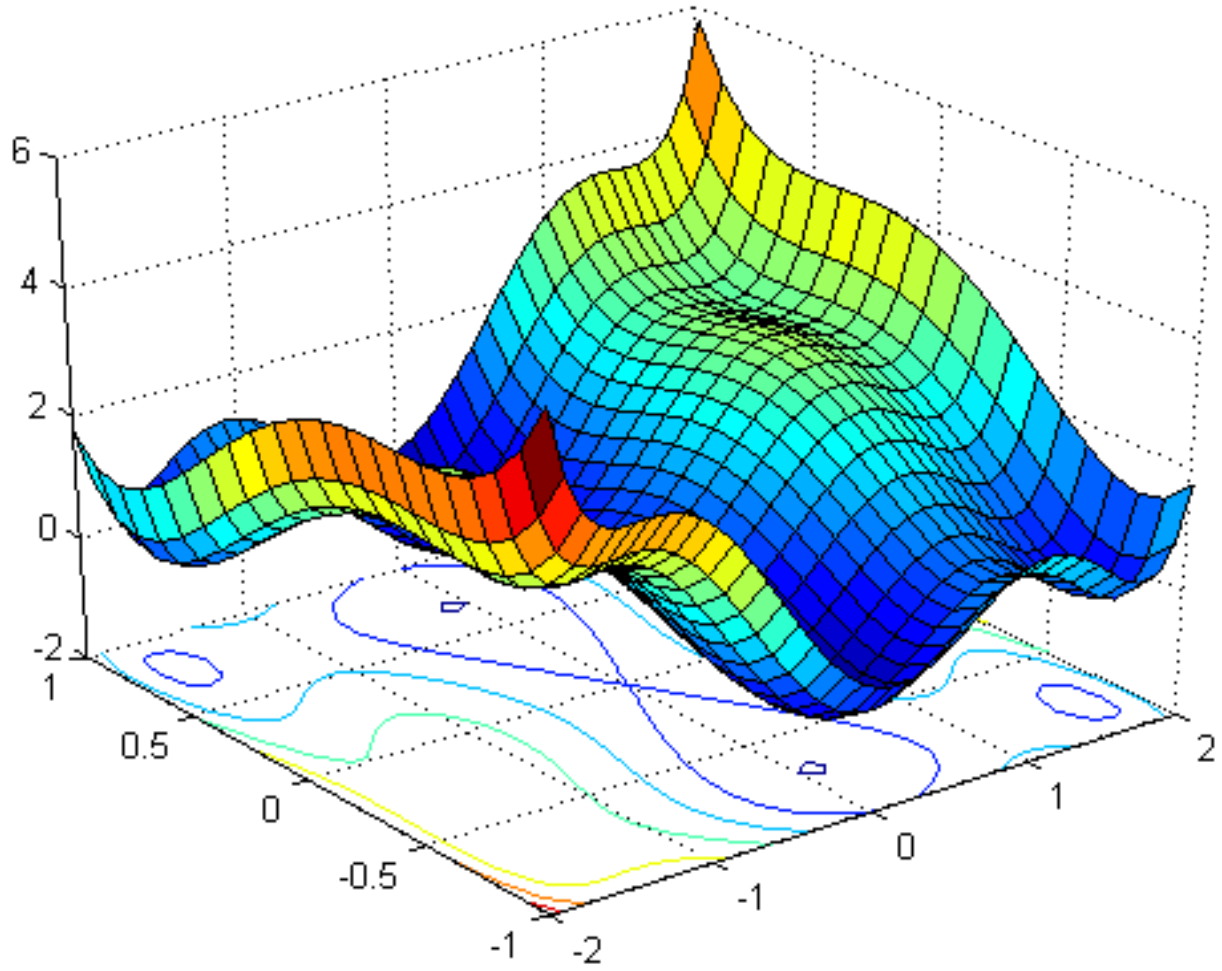
for $t = 1, 2, \ldots, T$

choose $\mathbf{v}_t$ at random from a distribution such that $\mathbb{E}[\mathbf{v}_t \mid \mathbf{w}^{(t)}] \in \partial f(\mathbf{w}^{(t)})$

update $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \mathbf{v}_t$

output $\bar{\mathbf{w}} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{w}^{(t)}$

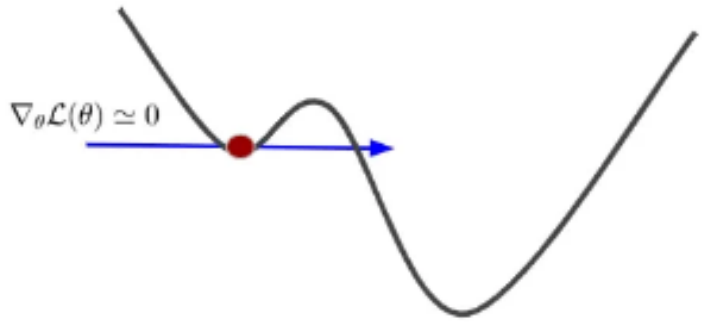
SGD & Local Minima



SGD & Non-Uniform Cost Function

When the **cost function is very irregular**, this can actually help the algorithm **jump out of local minima**

So, SGD has a **better chance** of finding the **global minimum** than Batch Gradient Descent in non-convex functions.



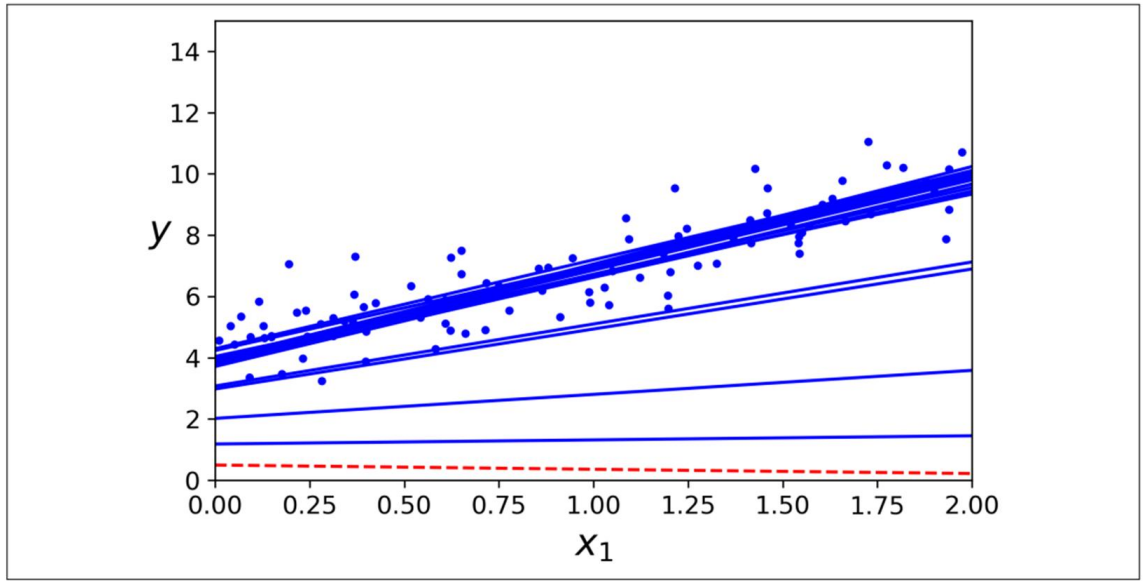
Therefore, randomness is good to **escape from local optima**, but bad because the algorithm can **never settle at the global minimum**.

One solution to this dilemma is **to reduce the learning rate gradually**.

SGD & Learning Rate

The steps start out large (which helps make quick progress and escape local minima), then get smaller and smaller, allowing the algorithm to settle at the global minimum

The function that determines the learning rate at each iteration is called the **learning schedule**.



SGD Observations

In SGD we do not require the update direction to be based exactly on the **gradient**.

Instead, we allow the **direction to be a random vector** and only require that its expected value at each iteration will equal the **gradient direction**.

SGD may not achieve global minimum but can be very close to it.

Gradual tuning of the Learning rate helps in getting very **close to global minimum**.

SGD is very effective in non-uniform cost functions and is helpful to get out when stuck at the local minima.

SGD Advantages & Disadvantages

The advantages of Stochastic Gradient Descent are:

- **Computationally efficient**. Each training step can be relatively fast, even with a huge training set.
- Easy to **implement** & **decreases overfitting** by focusing on only a portion of the training set at each step.

The disadvantages of Stochastic Gradient Descent include:

- SGD requires several hyperparameters, such as the **regularization** parameter and the number of **iterations**
- SGD is sensitive to **feature scaling**

Mini-Batch Gradient Descent

Mini-batch forms the **balance** between the **goodness** of gradient descent and the **speed** of SGD.

It considers **sampling a small number of data points** instead of just one point at each step.

Minibatch GD computes the gradients on **small random sets of instances** called **mini-batches**.

Instead of computing the gradients based on the **full training set** (as in Batch GD) **or** based on just **one instance** (as in Stochastic GD) it computes on **mini-batches**.

For the mini-batch gradient descent, we must divide our training set into **batches of size *n***

Advantages of Mini-Batch Gradient Descent

- **Computational Efficiency**: In terms of computational efficiency, this technique lies between the two previously introduced techniques.

- **Stable Convergence:** Another advantage is the more stable converge towards the global minimum since we calculate an average gradient over n samples that results in less noise.

- **Faster Learning**: As we perform weight updates more often than with stochastic gradient descent, in this case, we achieve a much faster learning process.

Disadvantages of Mini-Batch Gradient Descent

New Hyperparameter:

A disadvantage of this technique is the fact that in mini-batch gradient descent, a new **hyperparameter n**, called as the mini-batch size, is introduced.

It has been shown that the mini-batch size after the learning rate is the **second most important hyperparameter** for the overall performance of the neural network.

For this reason, it is necessary to **take time and try many different batch sizes** until a final batch size is found that works best with other parameters such as the learning rate.

Ex: 4, 8, 16, 32 ….

Summary

1. **Batch Gradient descent** can prevent the **noisiness of the gradient**, but we can get stuck in local minima and saddle points

2. With **Stochastic Gradient Descent** we have difficulties to settle on a global minimum, but usually, **don't get stuck in local minima**

3. The **Mini-batch** approach is the default method to implement the gradient descent algorithm in Deep Learning. It combines **all the advantages of other methods**, while not having their disadvantages

Classifiers

In machine learning, a **classifier** is an algorithm that automatically sorts or **categorizes data** into one or more "**classes**."

Targets, **labels**, and **categories** are all terms used to describe classes.

A **classifier** is used to train the Machine Learning model, and the model is then used to **classify new instances of data**.

Within Classifiers there are both **supervised** and **unsupervised classifiers**.

Unsupervised machine learning **classifiers** are fed just unlabeled datasets, which they sort into categories based on **pattern recognition**, **data structures**, and **anomalies**.

Classification Algorithms

Classification algorithms in machine learning map the input data into predefined categories.

There are different classifiers in Machine Learning. They are:

1. **Logistic Regression**
2. Support Vector Machines
3. **Naive Bayes Classifier**
4. Decision Tree
5. K-Nearest Neighbors (KNN)

Naive Bayes Classification

Naive Bayes models are a **group of extremely fast and simple classification algorithms** that are often suitable for **very high-dimensional datasets**.

Because they are **so fast** and have so **few tunable parameters**, they end up being very useful as a **quick baseline** for a classification problem.

Naive Bayes classifiers are built on **Bayesian classification methods**.

These rely on **Bayes's theorem**, which is an equation describing the relationship of **conditional probabilities** of statistical quantities.

The Naive Bayes assumes that each feature makes :

1. An **independent** contribution and
2. **Equal contribution** to the result or outcome.

Statistics

Population: The set that contains all data points from the dataset. The population size is denoted by *N*.

Sample: It is a randomly selected subset from the population — sample size is denoted by *n*.

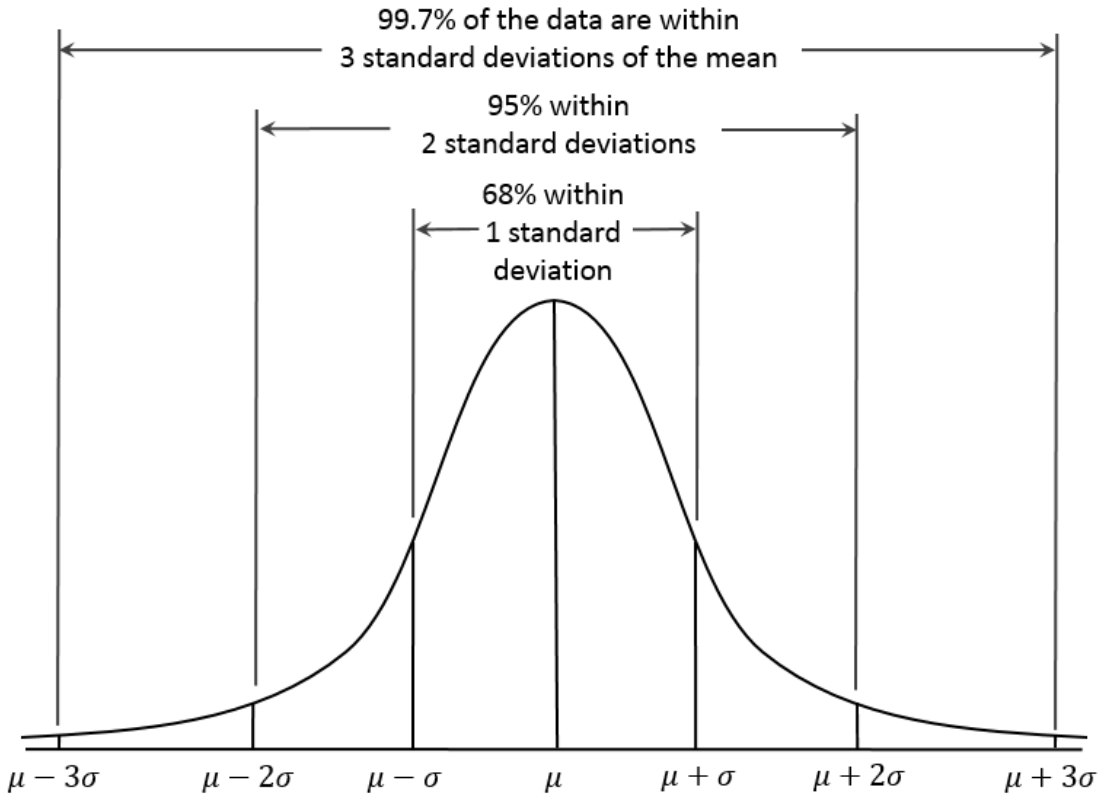
Distribution: It describes the data/population/sample range and how data is spread in that range.

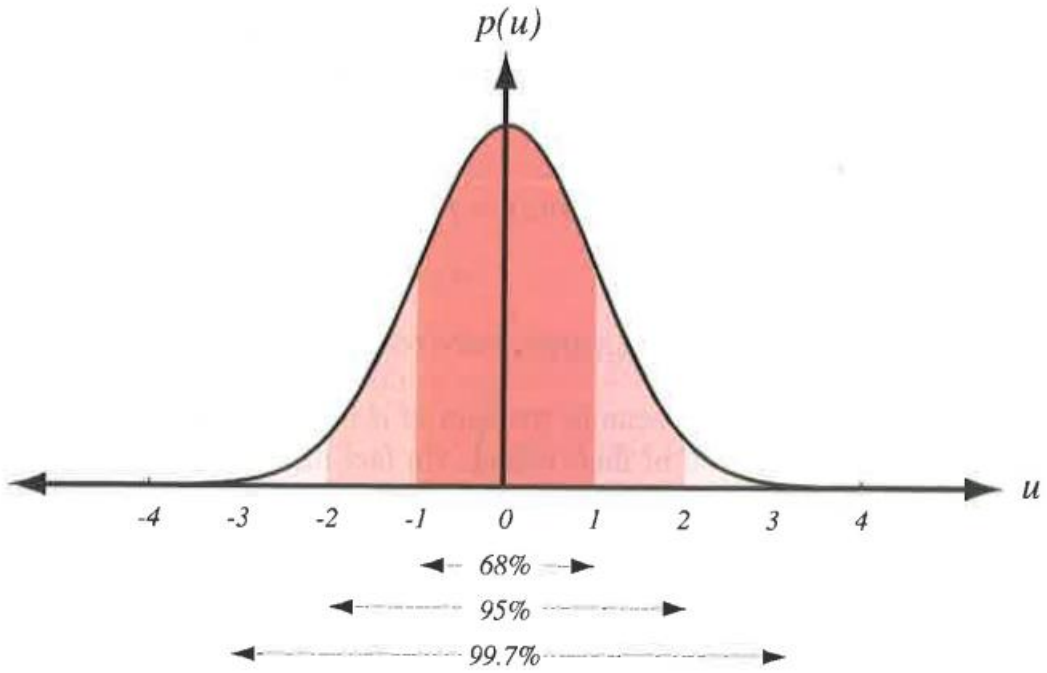
Mean: Average value of all data from the population or sample. The mean is denoted by μ (Mu) for populations and $\bar{x}$ (x bar) for samples.

Standard Deviation: Standard deviation is a measure of how spread the population is — denoted by σ (Sigma).

Data Distribution

Normal Distribution — Bell-Shaped Curve — Gaussian Distribution: When the population is spread perfectly symmetrical with **σ** standard deviations around the mean value, you get the following bell-shaped curve:





Mean & Standard Deviation

Mean $\mu = \frac{\sum_{i=1}^{N} x_i}{N} = \frac{x_1+x_2+x_3+\ldots+x_N}{N}$

Steps to compute the Standard Deviation

1. Calculate the **Mean**.
2. Subtract the **Mean from each value** to obtain deviations from the mean.
3. Square each of the **deviations**.
4. Add together all of these **squared deviations**. $\sigma = \sqrt{\frac{\sum (x-\mu)^2}{n}}$

Central Limit Theorem

The central limit theorem (CLT) states that the distribution of **sample means** approximates a **normal distribution** as the sample size gets larger.

Sample sizes **equal to or greater than 30** are considered sufficient for the CLT to hold.

A key aspect of CLT is that the **average of the sample means** and **standard deviations** will equal the population **mean** and **standard deviation**.

A **sufficiently large sample** size can predict the **characteristics of a population** accurately.

Bayes Classification

In Bayesian classification, we're interested in finding the **probability of a label** given some **observed features**, which we can write as **P(L | features).**

Bayes's theorem defines how to express this in terms of quantities we can compute more directly:

$$P(L \mid \text{features}) = \frac{P(\text{features} \mid L)P(L)}{P(\text{features})}$$

When trying to decide between two labels **L1** and **L2**—then one way to make this decision is to compute the ratio of the **posterior probabilities for each label**

$$\frac{P(L_1 \mid \text{features})}{P(L_2 \mid \text{features})} = \frac{P(\text{features} \mid L_1)}{P(\text{features} \mid L_2)} \frac{P(L_1)}{P(L_2)}$$

Naive Bayes Classification

In Naive Bayes, the **features are assumed to be independent of each other**, which is why it is called "naive".

This assumption simplifies the calculations and makes the **algorithm computationally efficient**.

However, it **may not hold true** in real-world scenarios, which can result in **lower accuracy**.

To **train a Naive Bayes model**, we first need to provide it with a **labeled dataset**, which consists of inputs and their corresponding labels.

Bayes Theorem

The Probability of '**B**' being true given that **'A'** is true **(Likelihood)**

The Probability of '**A**' being true **(prior)**

$$P(A|B) = \frac{P(B|A)\ P(A)}{P(B)}$$

The Probability of '**A**' being true given that **'B'** is true **(Posterior)**

The Probability of '**B**' being true **(Evidence)**

Naive Bayes Classification Steps

1. The model then **calculates the probability of each feature given each label**, based on the frequency of that feature in the dataset.

2. Next, it calculates the **prior probability of each label**, based on the frequency of each label in the dataset.

3. Once the model is trained, it can be used to predict the label of new inputs.

4. To do this, the model calculates the **posterior probability of each label given the features of the input**, using the Bayes theorem.

5. It then chooses the label with the **highest posterior probability** as the predicted label for the input.

Naive Bayes Classification – When to Use

Because naive Bayesian classifiers make such stringent assumptions about data, they will generally not perform well as a more complicated model.

Several advantages:

- They are **extremely fast** for both training and prediction
- They provide straightforward **probabilistic prediction**
- They are often very **easily interpretable**
- They have very **few** (if any) **tunable parameters**

These advantages mean a **Naive Bayesian classifier** is often a **good choice** as an **initial baseline** classification.

If it does not perform well, then we can begin exploring more sophisticated models, with some **baseline knowledge** of how well they should perform.

Naive Bayes Classification - Summary

Overall, Naive Bayes is a simple and effective algorithm that can be used for a variety of classification tasks.

Especially when the dataset has a **large number of features**.

However, its performance may be affected by the independence assumption and the **quality of the dataset** used to train it.

Performance Measures - Classification

Evaluating a classifier is often **significantly trickier** than evaluating a regressor.

Earlier in regression **Cross-Validation** was used as a good way to evaluate a model.

Accuracy is generally **not the preferred** performance measure for classifier.

Especially when you are dealing with **skewed datasets**.

For example:
Let us consider the dataset which has about 10% of the images of an 'Orange',

So, if we always guess that an image is **'not an Orange'**, we will be right about **90%** of the time.

Confusion Matrix

A better way to evaluate the performance of a classifier is to look at the **confusion matrix**.

A **Confusion Matrix** is a table that is used to define the **performance of a classification algorithm**.

A confusion matrix visualizes and summarizes the performance of a classification algorithm.

The general idea is to count the number of times instances of **class A** are misclassified as **class B**. **+** are misclassified as **-**

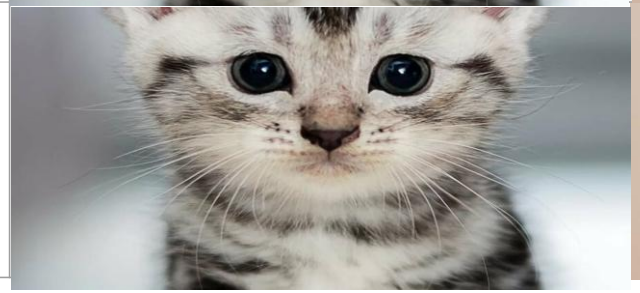
To compute the confusion matrix, you first need to have a set of **predictions**, so they can be compared to the **actual targets**.

Confusion Matrix - Table

	PREDICTED POSITIVE	PREDICTED NEGATIVE
ACTUAL POSITIVE	***TRUE POSITIVE***	***FALSE NEGATIVE (Type II error)***
ACTUAL NEGATIVE	***FALSE POSITIVE (Type I error)***	***TRUE NEGATIVE***

Each row in a confusion matrix represents an **actual class**, while each column represents a **predicted class**.

Confusion Matrix – Binary Classification Example

	PREDICTED VALUE POSITIVE (CAT)	PREDICTED VALUE NEGATIVE (DOG)
ACTUAL VALUE POSITIVE (CAT)		
ACTUAL VALUE NEGATIVE (DOG)		

The four possible outcomes for a binary image classification **Model** are:

1.True Positive - Image of Cat is predicted as Cat.

2.True Negative - Image of Dog is predicted as Dog.

3. False Positive or a **Type I Error -** Image of Dog is predicted as cat.

4. False Negative or a **Type II Error -** Image of Cat is predicted as Dog.

Precision

- An interesting one to look at is the accuracy of the positive predictions; this is called the precision of the classifier.
- Precision tells us when it predicts yes, how often is it correct. Precision should be high.

$$\text{precision} = \frac{TP}{TP + FP}$$

- TP is the number of **true positives**, and FP is the number of **false positives**.
- Precision is typically used along with another metric named recall, also called **sensitivity** or **true positive rate**

$$\text{recall} = \frac{TP}{TP + FN}$$

- FN is the number of **false negatives**

F1 Score & Specificity - Performance Measure

It is often convenient to combine **precision** and **recall** into a single metric called the **F1 score**, if you need a simple way to compare two classifiers.

The **F1 score** is the **harmonic mean** of **precision** and **recall**. Whereas the regular mean treats all values equally, the **harmonic mean** gives much more weight to low values.

As a result, the classifier will only get a high F1 score if both recall and precision are high.

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

Specificity determines the proportion of actual negatives that are correctly identified.

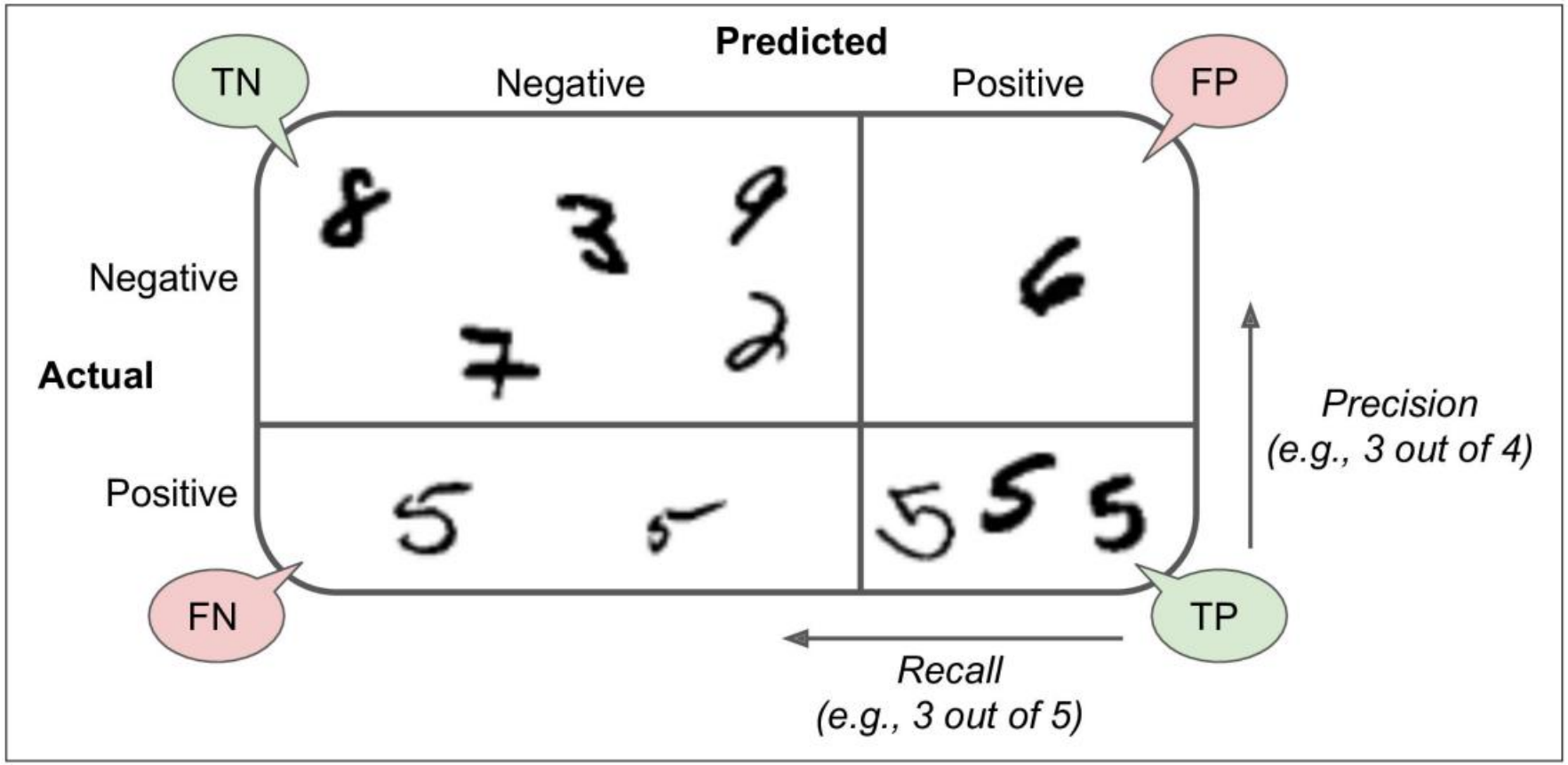
$$\textbf{\textit{Specificity}} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

Confusion Matrix – MNIST Example

Classification of the MNIST handwritten digits(0-9).

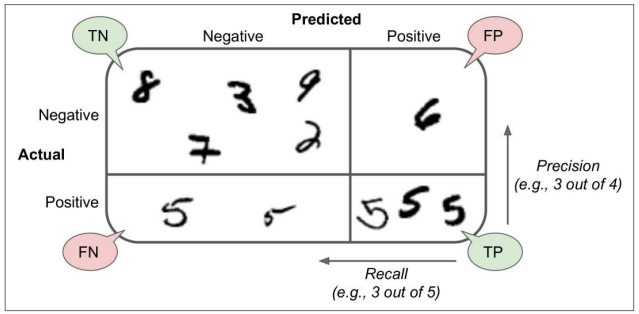
If a classifier has to identify the handwritten image as ‘5’, then there is a good chance for it to confuse with ‘3’.

Confusion Matrix – Classifying MNIST Digit '5'



Confusion Matrix - Classifying MNIST Digit '5'

```
[[53057,  1522],
 [ 1325,  4096]]
```



- The first row of this matrix considers **non-5 images** (**the negative class**): 53,057 of them were correctly classified as non-5s (**they are called true negatives**), while the remaining 1,522 were wrongly classified as 5s (**false positives**).

- The second row considers the images of 5s (**the positive class**): 1,325 were wrongly classified as non-5s (**false negatives**), while the remaining 4,096 were correctly classified as 5s (**true positives**).

- A perfect classifier would have only **true positives** and **true negatives**, and it would have nonzero values only on its main diagonal (top left to bottom right)

Evaluation Metrics - Classification

Metric	Formula	Interpretation
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Overall performance of model
Precision	$\frac{TP}{TP + FP}$	How accurate the positive predictions are
Recall Sensitivity	$\frac{TP}{TP + FN}$	Coverage of actual positive sample
Specificity	$\frac{TN}{TN + FP}$	Coverage of actual negative sample

$$\text{F1 Score} = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Receiver Operating Characteristic (ROC)

Receiver Operating Characteristic (ROC) is a graphical tool used in binary classification to evaluate the performance of a classifier.

It simplifies the evaluation of a classifier's ability to discriminate between two classes, typically denoted as positive (1) and negative (0) outcomes.

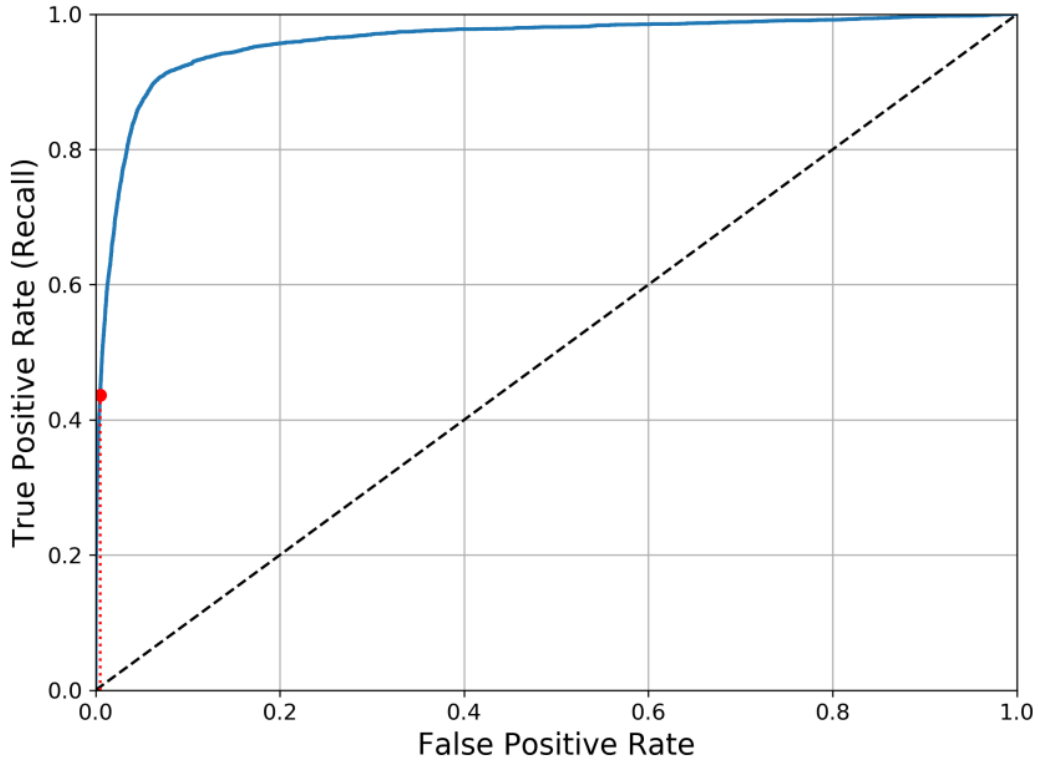
For example, in medical testing, positive may mean the presence of a disease, and negative may mean its absence.

True Positive Rate (TPR) or **Sensitivity**: It is the ratio of correctly predicted positive instances to all actual positive instances. TPR measures how well the classifier identifies positive cases.

False Positive Rate (FPR): It is the ratio of incorrectly predicted positive instances to all actual negative instances. FPR measures how often the classifier incorrectly identifies negative cases as positive.

Receiver Operating Characteristic (ROC)

The ROC curve is a graphical representation of the TPR (Sensitivity) against the FPR (1 - Specificity) as the classification threshold varies.



Each point on the ROC curve corresponds to a specific threshold used to classify data into positive or negative classes.

Area Under the Curve (AUC)

The **AUC** is a single scalar value that represents the overall performance of the classifier.

It indicates the probability that the classifier will rank a randomly chosen positive instance higher than a randomly chosen negative instance.

AUC is a concise metric summarizing the ROC curve's performance, with values ranging from 0 to 1, where 1 represents a perfect classifier.

ROC vs AUC

When to use ROC and AUC?

When evaluating the performance of a single binary classifier, we can use both ROC and AUC.

The ROC curve gives a visual representation of how the classifier performs at different thresholds, while the AUC provides a single metric summarizing the overall performance.

When comparing the performance of multiple classifiers, AUC is preferred as it offers a concise and standardized measure for easy comparison.

ROC is the best option when it is necessary to understand the classifier's behavior across various thresholds and also the trade-off between TPR and FPR.

Cross Entropy

Cross-entropy is a mathematical concept used in information theory and machine learning to measure the difference between two probability distributions.

Specifically, it quantifies how well a predicted probability distribution matches the true probability distribution.

Given two probability distributions P and Q, the cross-entropy H(P, Q) is defined as:

$$H(P, Q) = - \Sigma (P(i) * \log(Q(i)))$$

Where:
- P(i) is the probability of the i-th event according to the true distribution P.
- Q(i) is the probability of the i-th event according to the predicted distribution Q.
- The summation is taken over all events (i) for which P(i) > 0.

KL Divergence

The **Kullback-Leibler (KL) divergence**, is another concept from information theory and machine learning and is also known as relative entropy.

It measures the difference between two probability distributions P and Q.

The KL divergence $D_{KL}(P \mid\mid Q)$ between distributions P and Q is given by:

$$D_{KL}(P \mid\mid Q) = \Sigma\ (P(i) * \log(P(i) / Q(i)))$$

Where:
- P(i) is the probability of the i-th event according to the true distribution P.
- Q(i) is the probability of the i-th event according to the predicted/approximated distribution Q.
- The summation is taken over all events (i) for which P(i) > 0 and Q(i) > 0.

Cross Entropy vs KL Divergence

- The **Cross-Entropy** is a positive value and reaches its minimum when the two distributions perfectly match.
- It is widely used in various machine learning tasks, particularly in training classification models, where it serves as a loss function to be minimized during the learning process.
- Unlike the Cross-Entropy, the **KL divergence** is not symmetric (i.e., D_KL(P || Q) ≠ D_KL(Q || P)).
- It quantifies how much information is lost when using Q to approximate P, and it is non-negative (i.e., D_KL(P || Q) >= 0).
- The KL divergence is used in various machine learning tasks, such as in probabilistic models, to measure how well one distribution approximates another.

Checklist For a Machine Learning Project

1. Frame the problem and look at the big picture.
2. Get the data.
3. Explore the data to gain insights.
4. Prepare the data to better expose the underlying data patterns to Machine Learning algorithms.
5. Explore many different models and short-list the best ones.
6. Fine-tune your models and combine them into a great solution.
7. Present your solution. (Launch, monitor, and maintain your system)

END